

PORTABLE ZERO-ADDITIONAL-VRAM SELF-SPECULATIVE DECODING

Joint Layer-K Runtime Control under Dynamic Hardware and Resource Constraints

Phạm vi nghiên cứu: Efficient Computing for AI / Systems for AI / LLM Inference

Thiết bị đánh giá chính: NVIDIA GeForce RTX 4050 Laptop GPU, 6GB GDDR6

Vai trò của RTX 4050: một evaluation platform đại diện cho môi trường tài nguyên hạn chế, không phải giới hạn của bài toán nghiên cứu.

Mô hình phát triển: Qwen2.5-3B-Instruct; đối chứng Llama-3.2-3B-Instruct; mở rộng sang các checkpoint/model/hardware khác khi khả thi.

Mục tiêu paper: nghiên cứu khả năng điều khiển và chuyển giao chính sách self-speculative giữa các model, context và trạng thái phần cứng bằng calibration có ngân sách thấp, thay vì tối ưu riêng cho một GPU.

Lưu ý quan trọng: Phần dưới đây là lớp cập nhật mới được thêm vào proposal. Toàn bộ proposal gốc 17 trang được giữ nguyên ở cuối tài liệu dưới dạng Phụ lục A, không xóa nội dung.

0. ĐIỀU CHỈNH PHẠM VI NGHIÊN CỨU

Phiên bản trước đặt RTX 4050 6GB làm thiết bị mục tiêu. Sau khi đối chiếu thêm literature 2025-2026, cách framing đó quá hẹp cho một paper có tham vọng khái quát. Các công trình gần đây đã nghiên cứu hardware-aware self-speculation, device-aware calibration và adaptive layer selection; ví dụ KnapSpec dùng latency phụ thuộc phần cứng để chọn layer, còn UniSpec công bố một cơ chế calibration theo thiết bị và đánh giá trên nhiều hardware. [18][19]

Vì vậy, trong phiên bản này, RTX 4050 được giữ nguyên như một thiết bị phát triển và đánh giá chính, nhưng không còn là định nghĩa của bài toán. Đối tượng khoa học trở thành một target LLM M , một trạng thái workload x và một trạng thái phần cứng h ; hệ thống phải chọn cấu hình drafting phù hợp với trạng thái hiện tại và có thể thích nghi khi h thay đổi.

Phần core vẫn giữ nguyên các ý tưởng trong proposal gốc: training-free self-speculative decoding, layer skipping, exact verification, adaptive K , telemetry và tối ưu VRAM/energy. Thay đổi nằm ở cấp độ nghiên cứu: thay vì hỏi “cấu hình nào tốt nhất trên RTX 4050?”, paper hỏi “làm thế nào để một chính sách speculative có thể chuyển giao, tự hiệu chỉnh và duy trì Pareto efficiency khi model, context và hardware state thay đổi?”.

Nguyên tắc này cũng tránh một novelty claim không còn phù hợp. SpecBound đã giải quyết adaptive bounded self-speculation dựa trên confidence và bảo đảm exact equivalence; KnapSpec đã giải quyết adaptive layer selection theo hardware-specific latency; UniSpec đã có device-aware calibration cho draft size. [18][19][20] Contribution mới do đó phải nằm ở sự liên hợp giữa layer budget, K , hard constraints và low-budget cross-state adaptation, rồi được xác nhận bằng ablation và transfer experiments.

Bảng 0.1. So sánh framing cũ và framing mới

Thành phần	Framing cũ	Framing mới
Hardware	RTX 4050 6GB là target	RTX 4050 là evaluation platform; policy phải hướng tới portability
Action	Layer set + K	Joint action (S, K) dưới constraint
Adaptation	Runtime trong một thiết bị	Runtime + chuyển giao giữa hardware/model/context states
Objective	Throughput / VRAM / energy	Pareto efficiency + transfer efficiency + constraint stability
Calibration	Offline profiling trên thiết bị	Few-shot calibration cho hardware/state mới
Novelty	HECC trên laptop GPU	Candidate: portable constrained policy, joint Layer- K control, low-bud

Cụm từ “Zero-Additional-VRAM” vẫn được giữ. Nó có nghĩa là không nạp một draft model neural network độc lập; target weights, KV cache, scratch buffers và mọi runtime allocations vẫn tiêu thụ VRAM như bình thường.

1. FORMULATION TỔNG QUÁT CHO PAPER

Cho target LLM M có L transformer blocks. Ở decoding cycle t, hệ thống quan sát trạng thái model/workload và trạng thái phần cứng:

```
s_t = [h_model, c_t, entropy_t, acceptance_history_t, hardware_state_t]
```

Trong đó h_model mô tả các thuộc tính ổn định của checkpoint (số layer, hidden size, precision); c_t là context state; hardware_state_t chứa các telemetry có thể thay đổi như latency, utilization, power, temperature, available VRAM và TGP/power mode.

Action của controller là:

```
a_t = (S_t, K_t, mode_t)
```

với S_t là layer mask hoặc exit budget, K_t là speculative window và mode_t là mục tiêu vận hành như MAX-THROUGHPUT, MIN-ENERGY hoặc SAFE-MEMORY.

Thay vì một scalar objective cố định, paper nên xem xét tập Pareto của ba mục tiêu chính:

```
maximize {Throughput, -EnergyPerToken, -PeakVRAM} subject to Exactness = 1
```

và thêm các ràng buộc tùy workload:

- PeakVRAM $\leq$ V_budget.
- P95 TPOT $\leq$ L_budget.
- Power $\leq$ P_budget khi chế độ nguồn có thể kiểm soát.
- Greedy consistency = 100% ở chế độ greedy.
- Distributional correctness theo sampler exact khi sampling.

Điểm nghiên cứu mới không phải khẳng định rằng một heuristic cụ thể luôn tối ưu. Câu hỏi là liệu có thể xây một policy có độ ổn định và khả năng chuyển giao tốt khi trạng thái phần cứng thay đổi hay không.

1.1. Hardware-normalized cost model

Thay vì lưu tên GPU trong policy, hệ thống dùng một vector hardware signature:

```
h = [B_eff, F_eff, L_launch, V_free, P_limit, T_gpu, U_gpu]
```

Trong đó các đại lượng được đo hoặc ước lượng từ microbenchmark/telemetry. B_eff là effective memory bandwidth, F_eff là compute throughput thực nghiệm theo precision, L_launch biểu diễn overhead dispatch/synchronization, V_free là VRAM headroom và P_limit/T_gpu/U_gpu mô tả trạng thái năng lượng - nhiệt - utilization.

Mục tiêu không phải xây một mô hình phần cứng hoàn hảo. Một mô hình chi phí đủ tốt để dự đoán thứ tự ưu tiên giữa các action candidate là đủ cho controller.

2. ỨNG VIÊN ĐÓNG GÓP MỚI: PORTABLE CONSTRAINED SELF-SPECULATION

Tên tạm cho cơ chế: Portable Adaptive Constrained Self-Speculation (PACS). Đây chỉ là tên làm việc, chưa phải tên paper cuối cùng.

PACS coi layer budget và K là một quyết định liên hợp. Mỗi action candidate được đánh giá bằng một mô hình utility/cost ngắn hạn; controller cập nhật quyết định khi telemetry mới xuất hiện. Điểm khác biệt dự kiến không phải một tín hiệu mới riêng lẻ, mà là protocol cho phép policy thích nghi trong nhiều hardware state với ngân sách calibration nhỏ.

2.1. Ba lớp thích nghi

- **Model-aware:** CKA/cosine/KL, layer count, precision, model size.
- **Workload-aware:** entropy, acceptance history, context length, generation mode.
- **Hardware-state-aware:** draft/verify latency, VRAM headroom, power, temperature, utilization.

2.2. Low-budget calibration

Khi hệ thống gặp hardware/state mới, thay vì exhaustive search toàn bộ (S,K), controller chạy một tập microbenchmarks nhỏ để ước lượng cost surface. Với n calibration measurements, gọi U_{oracle} là utility của exhaustive search và U_n là utility của policy sau n phép đo:

$$\text{TransferEfficiency}(n) = U_n / U_{\text{oracle}}$$

và calibration regret:

$$\text{Regret}(n) = U_{\text{oracle}} - U_n$$

Đây là metric quan trọng mới của paper: không chỉ hỏi “method có nhanh không?”, mà hỏi “method có cần bao nhiêu phép đo để biết cách chạy tốt trên một môi trường mới?”.

2.3. Leave-one-hardware-state-out evaluation

Một protocol kiểm chứng khả năng khái quát là leave-one-state-out. Nếu có các môi trường A, B, C, D, policy được calibration trên A+B+C rồi test zero/few-shot trên D; lần lượt thay D bằng C, B và A. Với GPU thật khác nhau, phép kiểm chứng này là cross-device transfer. Với cùng RTX 4050 nhưng power/thermal state khác nhau, đó là cross-state transfer.

3. ĐỊNH VỊ SO VỚI LITERATURE 2024-2026

Literature mới cho thấy ý tưởng “hardware-aware” hoặc “adaptive K” đơn lẻ đã không còn đủ để tạo novelty. KnapSpec xây adaptive layer selection bằng knapsack và mô hình latency hardware/context-specific; SpecBound adaptive bounded self-speculation với layer-wise confidence calibration; SVIP dùng entropy để điều chỉnh draft length; UniSpec dùng device-aware calibration để chọn draft size và đánh giá trên nhiều hardware. [18]-[21]

Nhánh literature	Đã giải quyết	Khoảng còn mở cho proposal
KnapSpec	Hardware-specific latency + adaptive layer selection	Joint Layer-K control + transfer/constraint protocol
SpecBound	Confidence calibration + adaptive bound, exact self-spec	Multi-objective resource constraints + cross-state transfer
SVIP	Entropy-based dynamic draft length	Entropy chỉ là một state feature trong constrained controller
UniSpec	Device-aware calibration + hardware diversity	Low-budget transfer of joint (S,K), hard VRAM/power constraints và state transfer
Sequoia / hardware-aware	Hardware-aware optimization của tree/speculation	Self-spec layer budget + K + resource frontier
DuoDecoding	Hardware-aware draft budget trong CPU/GPU heterogeneous setting	Single-setting self-spec + portability under resource constraints

UniSpec là một đối chiếu bắt buộc trong phiên bản paper mới. Công trình này công bố một framework training-free, lossless, plug-and-play trên nhiều hardware và có device-aware calibration để chọn draft size; do đó proposal không được gọi “hardware portability” đơn thuần là novelty. Contribution candidate phải chứng minh thêm giá trị của joint Layer-K action, hard resource constraints, telemetry feedback và low-budget policy transfer. [18]

Các công trình trên cũng gợi ý rằng cách định lượng novelty nên là một tập câu hỏi thực nghiệm: calibration budget bao nhiêu là đủ, constraint violation rate là bao nhiêu, policy có giữ được Pareto efficiency không, và joint optimization có đóng góp độc lập sau ablation hay không.

4. THUẬT TOÁN ĐỀ XUẤT VÀ KIẾN TRÚC ĐIỀU KHIỂN

4.1. Candidate action space

Tạo một candidate set nhỏ thay vì cho controller tìm trên toàn bộ 2^L layer masks. Ví dụ:

$$A = \{(S_1, K_1), (S_1, K_2), \dots, (S_m, K_n)\}$$

Candidate masks được tạo từ static/random/CKA/cosine hoặc một knapsack-style selector; candidate K có thể là 1,2,4,6,8 hoặc các mức do runtime policy đề xuất. Điều này giúp controller tập trung vào runtime selection thay vì giải lại bài toán combinatorial ở mọi token.

4.2. Cost prediction

Với mỗi action a, ước lượng:

$$U(s, a) = w_T * E[\text{accepted_tokens/cycle}] - w_L * E[\text{cycle_latency}] - w_E * E[\text{energy/cycle}] - w_V * \text{Risk}(\text{VRAM}) - w_P * \text{Risk}(\text{power})$$

Cost model được cập nhật bằng telemetry thực. Ở phiên bản đầu, có thể dùng moving average + linear/ridge regression; không cần reinforcement learning ngay. Nếu controller đơn giản vẫn cho transfer tốt, kết quả đó dễ giải thích và dễ tái lập hơn.

4.3. Safety mechanism

- Hard rejection của action nếu predicted/observed peak VRAM vượt V_{budget} .
- Fallback về một cấu hình an toàn đã đo trước nếu controller quan sát instability.
- Hysteresis và cooldown để tránh controller oscillation.
- Separate correctness path: controller không được thay đổi exact verification semantics.

4.4. Runtime feedback loop

```
Observe -> predict cost -> reject unsafe actions -> select (S,K) -> draft -> verify ->
commit target KV -> update model -> next cycle.
```

Cấu trúc này giữ nguyên tinh thần HECC trong proposal gốc, nhưng nâng mục tiêu từ một GPU cụ thể thành một policy có thể hiệu chỉnh theo hardware state.

5. THIẾT KẾ THỰC NGHIỆM TỔNG QUÁT

5.1. Hardware tiers

- **Tier A - primary:** RTX 4050 Laptop 6GB. Đây là nền tảng phát triển, debug và stress-test memory-constrained.
- **Tier B - consumer/desktop:** một GPU consumer khác có compute/memory profile khác biệt.
- **Tier C - data-center/cloud:** một GPU có memory bandwidth/compute throughput cao hơn để kiểm tra transfer giữa hardware regimes.
- **Optional state shifts:** plugged/performance, reduced power, thermal throttling hoặc background-load states khi hệ điều hành/phần cứng cho phép đo hợp lệ.

Không cần có tất cả tier trên để bắt đầu. Điều kiện tối thiểu cho paper là RTX 4050 + ít nhất một hardware regime khác đủ khác biệt để kiểm tra transfer, hoặc một tập controlled hardware states trên cùng máy khi API cho phép.

5.2. Model and workload axes

Trục	Mức đề xuất
Model	Qwen2.5-3B-Instruct; Llama-3.2-3B-Instruct; optional 7B-class khi VRAM/cloud cho phép
Precision	4-bit target; FP16/BF16 cross-check ở một subset để tách numerical effects
Context	512, 2K, 4K và mức dài nhất chạy ổn định
Workload	General text, code, math/reasoning, long-context
Batch	1 là primary; batch nhỏ là robustness test
Generation	Fixed output budget + matched prompt protocol

5.3. Exactness protocol

Tách cache equivalence, greedy consistency và distributional correctness. Mismatch phải được ghi tại timestep đầu tiên, cùng logits top-1/top-2, layer mask, K, precision và cache state để phân biệt algorithmic error với numerical non-determinism.

6. METRICS MỚI CHO PAPER TỔNG QUÁT

Metric	Định nghĩa	Ý nghĩa
Speedup	$\text{Throughput_method} / \text{Throughput_vanilla}$	Hiệu năng end-to-end
Acceptance	accepted / drafted	Chất lượng draft
TPOT / TTFT	time/token và time/first-token	Interactive latency
Peak VRAM	allocated + reserved + NVML process memory	Hard resource constraint
Energy/token	J / generated token	Energy efficiency
Constraint violation rate	violations / cycles	Độ an toàn của controller
Transfer Efficiency	$U_{\text{policy}} / U_{\text{oracle}}$	Hiệu quả sau calibration
Calibration Regret	$U_{\text{oracle}} - U_{\text{policy}}$	Chi phí thích nghi trên hardware/state mới
Adaptation overhead	$\text{controller_time} / \text{generation_time}$	Chi phí của closed-loop control
Pareto hypervolume	diện tích/thể tích frontier chuẩn hóa	Chất lượng trade-off đa mục tiêu

6.1. Vì sao Transfer Efficiency quan trọng

Một phương pháp có thể đạt speedup cao trên một GPU nhưng cần exhaustive search riêng cho từng thiết bị. Trong một paper tổng quát, đó chưa phải là portability. Transfer Efficiency và Calibration Regret cho phép đo chính xác “chi phí để biết cách chạy tốt trên môi trường mới”.

6.2. Pareto evaluation

Không chỉ báo một số speedup. Với mỗi hardware state, vẽ frontier giữa throughput, energy/token và peak VRAM. So sánh frontier của PACS với KnapSpec, SpecBound, UniSpec và các baseline phù hợp. Claim mạnh chỉ được đưa ra nếu có vùng mà proposed policy đạt utility hoặc Pareto hypervolume tốt hơn dưới cùng constraint.

7. ABLATION VÀ CÂU CHUYỆN PAPER

7.1. Ablation bắt buộc

Ablation	Câu hỏi
No CKA/cosine	Representation signal có thực sự giúp action selection không?
No latency model	Nếu bỏ cost model và chỉ tối ưu acceptance thì end-to-end có xấu đi không?
Fixed K	Adaptive K đóng góp độc lập ở mức nào?
Fixed layer mask	Joint Layer-K có tốt hơn tối ưu từng biến riêng lẻ không?
No power term	Energy-awareness có thay đổi policy khi power/TGP thay đổi không?
No VRAM constraint	Throughput-only policy có gây peak VRAM risk không?
Open-loop	Offline/static policy kém hơn closed-loop ở state shifts đến đâu?
No transfer calibration	Zero-shot portability khác few-shot portability bao nhiêu?

7.2. Paper narrative đề xuất

Problem: Self-speculative decoding không còn là ý tưởng mới; hardware-aware selection và adaptive K cũng đã có literature mạnh. Vấn đề còn lại là runtime policy có thể cùng lúc duy trì correctness, throughput, VRAM safety và energy efficiency khi môi trường thay đổi hay không.

Method: joint action (S,K), hardware-state-aware cost model, few-shot calibration và closed-loop controller với hard constraints.

Evidence: cross-hardware / cross-state transfer, context sweep, model sweep, exactness tests, Pareto curves và ablation.

Claim discipline: candidate novelty only. Nếu UniSpec/KnapSpec/SpecBound đã giải một phần contribution mới, paper phải thu hẹp claim thay vì đổi tên cùng một kỹ thuật.

8. ROADMAP TRIỂN KHAI SAU KHI TỔNG QUÁT HÓA

Phần 14 tuần trong proposal gốc vẫn được giữ nguyên ở Phụ lục A. Bản cập nhật dưới đây là lớp paper-extension; có thể chạy song song sau khi correctness đạt.

Phase	Deliverable mới
Correctness foundation	KV cache, cache equivalence, exactness matrix, end-to-end tests
Baseline parity	KnapSpec, SpecBound, SVIP/appropriate adaptive baseline, UniSpec where reproducible
Single-device controller	Joint Layer-K controller + VRAM/power safety
Calibration engine	Microbenchmark sampler + cost-model update + calibration budget tracking
Transfer benchmark	Leave-one-hardware-out và cross-state evaluation
Pareto analysis	Throughput-energy-VRAM frontier + hypervolume
Robustness	Context/model/precision/power-state sweeps
Paper artifact	configs, seeds, manifests, plots, raw logs, exact checkpoint revision

8.1. Điều kiện dừng của research branch

- Nếu joint controller không vượt baseline ở bất kỳ vùng constraint nào, không được gọi đó là contribution chính.
- Nếu portability chỉ đạt được sau exhaustive search trên từng thiết bị, claim nên giảm từ “portable” xuống “hardware-aware tuning”.
- Nếu exactness chưa ổn định dưới precision đã chọn, phải tạm dừng speed benchmark để sửa correctness.
- Nếu results trên nhiều hardware không đồng nhất, có thể pivot sang finding về failure boundary hoặc numerical/constraint characterization.

8.2. Kết quả mà paper cần chứng minh

Không phải một con số speedup cố định. Paper cần chứng minh ít nhất một finding mới về trade-off hoặc transfer: ví dụ policy joint Layer-K đạt Pareto point tốt hơn baseline dưới một constraint, hoặc few-shot calibration đạt utility gần oracle với rất ít measurements và vẫn giữ exactness.

9. TÀI LIỆU CẬP NHẬT BỔ SUNG CHO PHIÊN BẢN TỔNG QUÁT

- [18] Do, T. D., Le, N.-K., & Nguyen, L.-M. (2026). UniSpec: Training-Free Speculative Decoding for Robust LLM Acceleration Across Languages and Hardware. ACL 2026, 6288-6310. <https://aclanthology.org/2026.acl-long.285/>
- [19] Cha, S., Kim, G., Han, D., Yang, T., & Han, I. (2026). KnapSpec: Self-Speculative Decoding via Adaptive Layer Selection as a Knapsack Problem. arXiv:2602.20217. <https://arxiv.org/abs/2602.20217>
- [20] Wen, Z., & Feng, Y. (2026). SpecBound: Adaptive Bounded Self-Speculation with Layer-wise Confidence Calibration. Findings of ACL 2026, 16309-16321. <https://aclanthology.org/2026.findings-acl.802/>
- [21] Chen, Z. et al. (2024). Sequoia: Scalable, Robust, and Hardware-aware Speculative Decoding. arXiv:2402.12374. <https://arxiv.org/abs/2402.12374>
- [22] Lv, K. et al. (2025). DuoDecoding: Hardware-aware Heterogeneous Speculative Decoding with Dynamic Multi-Sequence Drafting. arXiv:2503.00784. <https://arxiv.org/abs/2503.00784>
- [23] Zhang, Z. et al. (2025). Draft Model Knows When to Stop: Self-Verification Speculative Decoding for Long-Form Generation. EMNLP 2025, 16685-16697. <https://aclanthology.org/2025.emnlp-main.844/>
- [24] Zhuge, X. et al. (2025). SpecOffload: Unlocking Latent GPU Capacity for LLM Inference on Resource-Constrained Devices. arXiv:2505.10259. <https://arxiv.org/abs/2505.10259>

Các tài liệu [1]-[17] và toàn bộ nội dung proposal gốc được giữ nguyên trong Phụ lục A. Các tài liệu [18]-[24] được bổ sung cho phiên bản tổng quát hóa và phải được đối chiếu lại ngay trước khi submission vì literature 2026 tiếp tục thay đổi.

10. KẾT LUẬN PHIÊN BẢN TỔNG QUÁT HÓA

Proposal vẫn giữ lõi kỹ thuật Zero-Additional-VRAM Self-Speculative Decoding trên resource-constrained GPU, nhưng định nghĩa lại contribution ở cấp độ paper: không tối ưu cho một GPU duy nhất, mà nghiên cứu cách một policy joint Layer-K có thể tự hiệu chỉnh và duy trì tính hiệu quả dưới các trạng thái hardware/workload khác nhau.

RTX 4050 6GB vì vậy vẫn rất quan trọng: nó là môi trường phát triển có áp lực VRAM thật, giúp debug cache, exactness, profiler và controller. Nhưng kết luận khoa học cuối cùng phải dựa trên cross-device/cross-state evidence, chứ không thể suy ra từ RTX 4050 đơn lẻ.

UniSpec làm cho “hardware portability” đơn thuần không còn là novelty. KnapSpec và SpecBound cũng loại bỏ novelty đơn giản của “hardware-aware layer selection” và “adaptive K”. Do đó, proposal chỉ nên chốt contribution nếu thực nghiệm cho thấy giá trị độc lập của joint Layer-K constrained control + low-budget calibration/transfer + Pareto resource management. Đây là candidate contribution có thể bác bỏ bằng thực nghiệm, và chính khả năng bác bỏ đó làm research question rõ ràng.

Sau trang này là toàn bộ proposal 17 trang gốc được giữ nguyên để bảo đảm không mất bất kỳ nội dung nào từ tài liệu đã cung cấp.

PHỤ LỤC A - TOÀN BỘ PROPOSAL GỐC ĐƯỢC GIỮ NGUYÊN

Các trang tiếp theo là bản PDF gốc, được chèn nguyên trạng, không biên tập hoặc xóa nội dung.

ĐỀ CƯƠNG NGHIÊN CỨU KHOA HỌC

ZERO-ADDITIONAL-VRAM SELF-SPECULATIVE DECODING

VỚI ĐIỀU KHIỂN THÍCH ỨNG NHẬN THỨC - PHẦN CỨNG CHO SUY LUẬN LLM TRÊN GPU
6GB

Bản đề cương dùng cho trao đổi với giáo sư và phát triển thành bài báo khoa học

Thông tin	Đề xuất
Lĩnh vực	Efficient Computing for AI / Systems for AI / LLM Inference
Thiết bị mục tiêu	NVIDIA GeForce RTX 4050 Laptop GPU, 6GB GDDR6
Mô hình chính	Qwen2.5-3B-Instruct; đối chứng Llama-3.2-3B-Instruct
Định hướng phương pháp	Training-free self-speculative decoding + adaptive layer selection + adaptive K + constrained runtime controller
Mục tiêu học thuật	Khóa luận/đồ án có khả năng phát triển thành paper thực nghiệm hệ thống
Lưu ý về tính mới Bản đề cương cố ý không khẳng định “novelty tuyệt đối”. Đến năm 2026, self-speculative decoding, adaptive layer selection và adaptive speculation length đã có nhiều công trình mạnh. Đóng góp đề xuất ở đây là một kiến trúc điều khiển vòng kín tối ưu dưới ràng buộc đồng thời về latency, peak VRAM và năng lượng trên GPU laptop, với mục tiêu được xác nhận bằng benchmark và ablation thực nghiệm.	

MỤC LỤC

Tóm tắt

1. Tổng quan và đặt vấn đề nghiên cứu
2. Tổng quan nghiên cứu liên quan
3. Phát biểu bài toán
4. Câu hỏi nghiên cứu và giả thuyết
5. Đóng góp dự kiến
6. Kiến trúc hệ thống đề xuất
7. Thuật toán lựa chọn layer
8. Điều chỉnh speculative window K
9. Hardware- and Energy-aware Constrained Controller (HECC)
10. Thiết lập thực nghiệm
11. Baseline và đối chiếu
12. Hệ thống chỉ số đánh giá
13. Ablation study
14. Kết quả kỳ vọng và tiêu chí thành công
15. Phân tích rủi ro kỹ thuật
16. Kế hoạch triển khai 14 tuần
17. Kiến trúc mã nguồn dự kiến
18. Lộ trình phát triển thành bài báo
19. Validation cho exactness và độ đúng
20. Giới hạn và phạm vi
21. Sản phẩm đầu ra
22. Tài liệu tham khảo chính
23. Kết luận đề cương

TÓM TẮT

Suy luận tự hồi quy của các mô hình ngôn ngữ lớn (Large Language Models - LLMs) thường bị giới hạn bởi chi phí truyền dữ liệu và việc phải thực hiện nhiều bước giải mã tuần tự. Speculative decoding giảm số lần target model phải thực hiện bằng cách sinh một chuỗi token ứng viên bằng mô hình nháp rồi xác thực song song bằng mô hình đích. Tuy nhiên, draft model độc lập tạo thêm nhu cầu về bộ nhớ, đặc biệt đáng chú ý trên GPU laptop có dung lượng VRAM 6GB. Một hướng thay thế là self-speculative decoding, trong đó chính target model được rút gọn trong pha drafting thông qua layer skipping, sau đó dùng toàn bộ mạng để verification. Draft & Verify và LayerSkip đã chứng minh tính khả thi của hướng này, trong khi các công trình mới hơn như CLaSp, DEL, SVIP, BanditSpec, CAS-Spec, KnapSpec và SpecBound cho thấy bài toán chọn layer và chọn speculative length đang tiến rất nhanh.

Đề tài này phát triển một kiến trúc Zero-Additional-VRAM Self-Speculative Decoding cho GPU 6GB. “Zero-additional-VRAM” được hiểu là không cần nạp một draft model neural network độc lập; hệ thống vẫn sử dụng VRAM cho target model, KV cache và các buffer thực thi cần thiết. Trọng tâm nghiên cứu gồm ba thành phần: (i) lựa chọn layer training-free dựa trên độ tương đồng biểu diễn và chi phí phần cứng; (ii) điều chỉnh speculative window K theo uncertainty và tín hiệu acceptance; và (iii) một Hardware- and Energy-Aware Constrained Controller (HECC) thực hiện điều khiển vòng kín dựa trên telemetry runtime, tối ưu throughput dưới ràng buộc peak VRAM và công suất/energy budget của thiết bị.

Đề tài không giả định trước tốc độ hoặc acceptance rate. Các con số được trình bày trong tài liệu chỉ là hypothesis/engineering targets. Tính đúng của giải mã được đánh giá tách biệt thành exact distributional correctness khi sampling và greedy consistency khi greedy decoding. Hệ thống sẽ được so sánh với vanilla autoregressive decoding, static layer skipping, random skipping và các baseline self-speculative/adaptive gần nhất có mã nguồn hoặc mô tả công khai phù hợp. Mục tiêu cuối cùng là xây dựng một pipeline có thể tái lập, đo được wall-clock speedup, TTFT, TPOT, acceptance, peak VRAM, additional VRAM, energy/token và GPU utilization trên RTX 4050 6GB.

Từ khóa: LLM inference, speculative decoding, self-speculative decoding, layer skipping, adaptive speculation, memory-constrained inference, GPU systems, energy-aware inference.

1. TỔNG QUAN VÀ ĐẶT VẤN ĐỀ NGHIÊN CỨU

1.1. Suy luận tự hồi quy và nút thắt bộ nhớ

Với một decoder-only Transformer, chuỗi xác suất có dạng:

$$P(x_1, \dots, x_N) = \prod_{t=1..N} P(x_t \mid x_1, \dots, x_{t-1})$$

Mỗi token mới phụ thuộc token trước đó nên việc giải mã có phụ thuộc tuần tự ở cấp token. Khi batch nhỏ, đặc biệt batch=1, thời gian của một bước decode có thể bị giới hạn bởi việc đọc trọng số và truy cập KV cache thay vì bởi năng lực FLOPs tối đa của GPU. Vì vậy, chỉ tăng năng lực compute không đảm bảo tăng tốc tương ứng; điều quan trọng là giảm số lần phải quét toàn bộ trọng số và cải thiện mức độ tận dụng băng thông.

RTX 4050 Laptop GPU có 6GB GDDR6, 2.560 CUDA cores và giao diện bộ nhớ 96-bit; NVIDIA công bố dòng 4050 laptop với TGP có thể thay đổi theo thiết kế laptop. Bảng thông danh định của phiên bản

4050 laptop thường được công bố là 192 GB/s. Do đó, thiết bị này là một nền tảng phù hợp để nghiên cứu các kỹ thuật giảm overhead bộ nhớ và giảm số vòng forward trong điều kiện resource-constrained. [1]

Điều chỉnh lập luận arithmetic intensity

Không sử dụng công thức “1 FLOP/Byte” như một hằng số của mọi LLM decode. Arithmetic intensity thực tế phụ thuộc batch size, context length, precision, KV cache, kernel và mức reuse. Đề tài sẽ dùng roofline-style analysis và đo hardware counter/telemetry thay vì tuyên bố một giá trị cố định.

1.2. Hạn chế của speculative decoding truyền thống trên GPU 6GB

Speculative decoding gốc tạo chuỗi draft bằng một mô hình phụ q và xác thực bằng target model p . Leviathan et al. chứng minh rằng quy trình này có thể giữ nguyên phân phối output trong khi giảm số lần chạy tuần tự của target. [2]

$$\text{acceptance}(x) = \min(1, p(x)/q(x))$$

Trong thiết bị 6GB, draft model độc lập làm tăng footprint. Với mô hình quantized, vấn đề không chỉ là trọng số mà còn là KV cache, workspace của kernel, activation và fragmentation của allocator. Do đó, một mục tiêu có giá trị thực tiễn là loại bỏ draft model độc lập mà vẫn giữ được lợi ích của verification song song.

1.3. Self-speculative decoding

Draft & Verify cho thấy target model có thể tự tạo draft bằng cách bỏ qua một tập layer trung gian khi drafting, sau đó khôi phục toàn bộ mạng ở bước verification. Công trình báo cáo speedup tới 1.99x trong các thiết lập của họ và nhấn mạnh rằng phương pháp không cần model phụ hay training thêm. [3] LayerSkip phát triển self-speculative theo hướng early exit nhưng yêu cầu training recipe đặc biệt để làm cho các layer sớm có chất lượng draft tốt hơn; họ báo cáo speedup cao trên CNN/DM, coding và semantic parsing. [4]

Điểm quan trọng đối với đề tài là: self-speculative decoding không còn là khoảng trống nghiên cứu sơ khai. Vì vậy, đề tài không thể coi “dùng chính model làm draft” là novelty. Cần nghiên cứu câu hỏi sâu hơn: cách lựa chọn cấu hình draft nào tốt nhất trên phần cứng cụ thể, thay đổi theo context thế nào, và làm sao tối ưu đồng thời chất lượng draft, latency, VRAM và năng lượng.

2. TỔNG QUAN NGHIÊN CỨU LIÊN QUAN

2.1. Speculative decoding cổ điển

Leviathan et al. đặt nền tảng cho exact speculative decoding: một mô hình nhanh hơn sinh ứng viên; target model xác thực nhiều token trong một lượt; rejection/resampling bảo toàn phân phối. [2] Đây là cơ sở lý thuyết của phần exactness trong đề tài.

2.2. Self-speculative và layer skipping

Draft & Verify [3] dùng layer skipping training-free, trong khi LayerSkip [4] kết hợp layer dropout và early-exit training. CLaSp [5] đưa layer selection theo in-context dynamic programming, cho phép điều chỉnh chiến lược sau các verification stage. DEL [6] chọn động exit layer và speculation length, còn SpecBound [10] xử lý vấn đề overconfidence của layer nông và adaptive bounds cho

speculation length. Những kết quả này cho thấy việc chọn layer/K động đã là một mạch nghiên cứu rõ ràng.

2.3. Adaptive speculative length và runtime adaptation

SVIP [7] chứng minh rằng draft length có thể điều chỉnh training-free dựa trên entropy của draft. BanditSpec [8] xem cấu hình speculative decoding như một bài toán online learning/bandit để thích nghi theo context. Một công trình ACL 2025 về on-the-fly adaptation [9] cũng nhấn mạnh ảnh hưởng đồng thời của task, hardware và model combination đến cấu hình tối ưu.

2.4. Hardware-aware và resource-aware hướng mới

KnapSpec [11] là một công trình đặc biệt quan trọng đối với đề tài hiện tại: họ mô hình hóa latency hardware-specific của Attention và MLP, rồi giải bài toán lựa chọn layer như knapsack để tối ưu tokens-per-time theo context length. Vì vậy, đề tài không được claim “hardware-aware adaptive layer selection” là khoảng trống trống hoàn toàn. KnapSpec cũng đề cập quan hệ giữa hidden-state similarity và token acceptance. [11]

CAS-Spec [12] mở rộng self-speculative theo hướng cascade và dynamically switchable inference strategies; Nightjar [13] nghiên cứu resource-aware speculative decoding trong bối cảnh serving và load variation. Những công trình này cho thấy “runtime adaptation” đơn thuần cũng không còn đủ mới.

2.5. Khoảng trống nghiên cứu khả thi cho đề tài

Khoảng trống/điểm chưa tối ưu	Câu hỏi đề tài
Tối ưu đa mục tiêu dưới ràng buộc thiết bị cá nhân	Có thể chọn cấu hình draft tối ưu throughput nhưng vẫn giữ peak VRAM và energy/token dưới ngưỡng thiết bị?
Điều khiển vòng kín theo telemetry thật	Acceptance, latency, VRAM và power có thể được dùng như phản hồi để cập nhật chính sách mà không cần offline benchmark toàn diện không?
Trade-off layer budget - K - context length	Một policy có thể phân bổ ngân sách giữa “độ nông của draft” và “độ dài draft” để tối ưu end-to-end wall-clock thay vì tối ưu từng biến riêng lẻ?
Energy-aware inference trên laptop GPU	Tối ưu tốc độ có luôn tối ưu J/token không, đặc biệt khi TGP/throttling thay đổi?
Đánh giá dưới ràng buộc VRAM cứng	Khi peak VRAM gần 6GB, policy nên hy sinh acceptance hay K để tránh OOM/allocator fragmentation như thế nào?

3. PHÁT BIỂU BÀI TOÁN

Cho một target LLM M có L transformer blocks và thiết bị G có tài nguyên hữu hạn. Tại mỗi chu kỳ speculative decoding t, hệ thống cần chọn một cấu hình draft S_t và độ dài K_t . Cấu hình S_t có thể được biểu diễn bởi tập layer giữ lại hoặc điểm exit tương đương. Mục tiêu là tối ưu hiệu năng end-to-end trong khi không dùng draft model độc lập và giữ exactness của output sau verification.

maximize $U_t = w_T * \text{Throughput}_t - w_L * \text{Latency}_t - w_E * \text{EnergyPerToken}_t - w_V * \text{VRAMPenalty}_t$
 subject to $\text{PeakVRAM}_t \leq V_{\text{budget}}, P95_TPOT_t \leq L_{\text{budget}}, \text{Exactness} = 1$

Ở đây U_t là utility thực dụng, không phải loss dùng để train model. Controller sẽ lựa chọn giữa một tập cấu hình đã được profile nhẹ hoặc cập nhật online. VRAMPenalty có thể bằng 0 khi dưới ngưỡng an toàn và tăng nhanh khi allocator usage tiến gần hard limit. Mục tiêu là biến bài toán “layer selection + K selection” thành một bài toán điều khiển constrained runtime.

4. CÂU HỎI NGHIÊN CỨU VÀ GIẢ THUYẾT

RQ1 - Representation/Layer selection

Làm thế nào để chọn các layer giữ lại cho draft sao cho acceptance tăng mà chi phí draft giảm?

RQ2 - Joint K and layer budget

Liệu việc tối ưu đồng thời layer budget và K có tốt hơn việc tối ưu từng biến độc lập?

RQ3 - Runtime control

Một controller dùng acceptance, entropy, context length, latency, VRAM và power có thể cải thiện wall-clock throughput so với fixed policy không?

RQ4 - Resource constraint

Khi VRAM headroom nhỏ hoặc power budget thay đổi, policy có thể duy trì tính ổn định và tránh OOM không?

RQ5 - Generalization

Hiệu quả có giữ được trên ít nhất hai kiến trúc 3B và nhiều kiểu workload không?

Giả thuyết	Phát biểu kiểm chứng
H1	CKA/cosine similarity ở hidden states tương quan dương với acceptance nhưng không đủ để tối ưu wall-clock nếu bỏ qua latency thực tế.
H2	Joint selection của layer budget và K đạt utility cao hơn fixed-K hoặc fixed-layer policies dưới cùng budget.
H3	Closed-loop controller có thể giảm TPOT và/hoặc J/token so với policy tĩnh trong ít nhất một phần workload, đồng thời không làm tăng peak VRAM vượt budget.
H4	Trong long-context, layer cost profile phụ thuộc context length đủ mạnh để static layer set suy giảm tương đối so với dynamic policy.
H5	Exact verification giữ greedy consistency 100% trong chế độ greedy và distributional correctness theo cơ chế rejection sampling khi sampling được thực hiện đúng.

5. ĐÓNG GÓP DỰ KIẾN

Đề tài phân biệt rõ “đóng góp nền tảng” và “đóng góp ứng viên mới”. Những phần đã được nghiên cứu trong literature sẽ được implement làm baseline hoặc thành phần tham chiếu; phần mới chỉ được coi là contribution sau khi có ablation và so sánh công bằng.

5.1. Đóng góp nền tảng được kế thừa/kiểm chứng

- Training-free self-speculative decoding bằng layer skipping, theo tinh thần Draft & Verify [3].
- Exact verification và rejection sampling theo speculative decoding [2].
- Layer similarity profiling bằng CKA/cosine/KL như tín hiệu lựa chọn.
- Adaptive K theo uncertainty/acceptance, đối chiếu với SVIP và các adaptive baselines [7].

5.2. Đóng góp ứng viên mới: HECC - Hardware- and Energy-aware Constrained Controller

HECC là thành phần trung tâm được đề xuất. Thay vì tối ưu riêng layer selection hoặc K, controller quan sát trạng thái hệ thống sau mỗi speculative cycle: acceptance rate trung bình gần nhất, entropy/uncertainty, context length, draft/verify latency, peak allocated/reserved VRAM, GPU utilization, power và energy/token. Từ trạng thái này, controller chọn một action từ tập cấu hình ứng viên gồm layer mask/exit budget và K.

$$\text{state}_t = [\text{entropy}_t, \text{acc_hist}_t, \text{context}_t, t_{\text{draft}}, t_{\text{verify}}, \text{VRAM}_t, \text{power}_t, \text{TGP}_t]$$

$$\text{action}_t = (\text{layer_budget}_t, K_t)$$

$$\text{action}^* = \underset{a}{\operatorname{argmax}} U(\text{state}_t, a) \text{ subject to } \text{VRAM}(a) \leq \text{budget}$$

Điểm khác biệt dự kiến không nằm ở từng tín hiệu riêng lẻ mà ở cách kết hợp chúng thành một constrained controller phù hợp với GPU laptop. Kết quả phải chứng minh rằng policy đa mục tiêu này tạo ra lợi ích wall-clock/energy thực tế mà các heuristic đơn mục tiêu không đạt được.

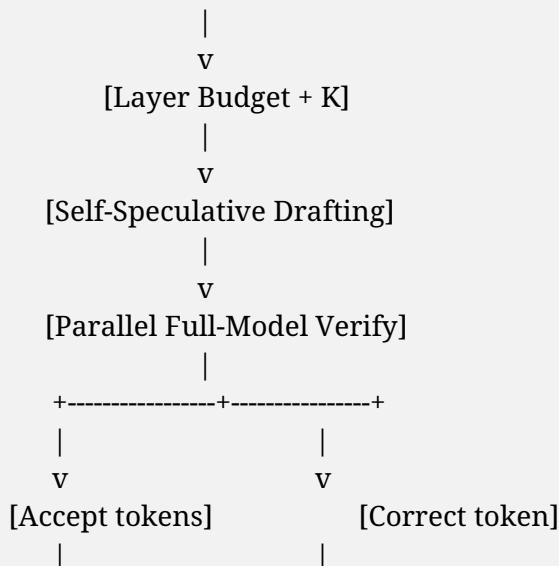
5.3. Một hướng phát triển paper mạnh hơn: Pareto Runtime Frontier

Hệ thống sẽ xây dựng một frontier thực nghiệm giữa ba mục tiêu: throughput, energy/token và peak VRAM. Thay vì trả về một cấu hình tối ưu duy nhất, controller duy trì tập các cấu hình Pareto-dominant. Khi thiết bị bị giới hạn power hoặc VRAM, policy có thể chuyển vùng hoạt động trên frontier. Đây là cách biến thí nghiệm từ “đo speedup” thành một nghiên cứu hệ thống về trade-off.

6. KIẾN TRÚC HỆ THỐNG ĐỀ XUẤT

Pipeline tổng thể

[Prompt / Context] -> [Target LLM state] -> [Runtime Telemetry] -> [HECC Controller]





6.1. Pha drafting

Target model được chạy theo một cấu hình draft S. S có thể là danh sách các transformer blocks giữ lại hoặc một exit-depth policy. Các layer bị bỏ qua không được xem là “mất thông tin vĩnh viễn”; draft chỉ là một phép xấp xỉ tạm thời. Candidate tokens được sinh tuần tự tối đa K bước hoặc dừng sớm nếu controller quyết định speculative window ngắn.

6.2. Pha verification

Toàn bộ target network được chạy một lượt cho candidate sequence. Tại đây, output distribution p của target được dùng để xác thực. Những token được chấp thuận sẽ được commit vào chuỗi; khi có rejection, chu kỳ xử lý đúng theo thuật toán speculative sampling/verification được chọn. Target KV cache sau verification là trạng thái chuẩn. Draft state được coi là ephemeral và có thể giải phóng sau cycle.

6.3. Quản lý KV cache

Thiết kế tránh yêu cầu “đồng bộ hai KV cache đầy đủ” giữa draft và target. Draft state chỉ cần đủ cho quá trình drafting; target KV cache là canonical state. Nếu cần lưu scratch buffers, kích thước tối đa phải được profile trước và đưa vào peak VRAM budget. Đây là điểm thực dụng hơn so với thiết kế pointer abstraction quá phức tạp.

6.4. Exactness

Với token ứng viên $\tilde{x}$ sinh từ q và target distribution p , acceptance được tính theo:

$$\alpha(\tilde{x}) = \min(1, p(\tilde{x}) / q(\tilde{x}))$$

Nếu bị từ chối, residual distribution được xây dựng từ phần dương của $p-q$. Với implementation chính xác và cùng điều kiện sampling, cơ chế này bảo toàn phân phối mục tiêu. Đề tài sẽ phân biệt hai kiểm chứng: (i) greedy consistency - token sequence có trùng với vanilla greedy hay không; (ii) distributional correctness - các kiểm định phân phối dưới sampling.

$$p_{\text{res}}(x) = \max(0, p(x)-q(x)) / \sum_v \max(0, p(v)-q(v))$$

7. THUẬT TOÁN LỰA CHỌN LAYER

7.1. Offline profiling nhẹ

Một calibration set nhỏ được dùng để thu hidden states H_i của từng layer. Các tín hiệu ban đầu gồm CKA, cosine similarity và sự thay đổi logit/KL ở output. Không sử dụng Hessian toàn phần ở phiên bản đầu vì chi phí và độ phức tạp có thể làm lệch trọng tâm khỏi hệ thống inference.

$$\text{CKA}(K,L) = \text{HSIC}(K,L) / \sqrt{\text{HSIC}(K,K) * \text{HSIC}(L,L)}$$

Từ profiling này tạo candidate masks $S_1 \dots S_m$. Mỗi mask được benchmark ngắn trên đúng GPU mục tiêu để lấy latency draft thực tế. Như vậy similarity là tín hiệu chất lượng; hardware measurement là tín hiệu chi phí.

7.2. Scoring layer budget

$$\text{Score}(S) = a * \text{Sim}(S) - b * \text{Cost}(S | \text{context}) - c * \text{VRAM}(S)$$

Trong đó Sim có thể kết hợp CKA/cosine/KL; Cost được lấy từ profiler hoặc model latency; VRAM là peak scratch footprint. Candidate set sau đó được đưa vào HECC thay vì chọn một mask cố định duy nhất.

7.3. Vì sao không dùng CKA đơn độc?

CKA cao không đồng nghĩa acceptance cao và càng không đồng nghĩa wall-clock tốt. Một layer có similarity cao nhưng chi phí vẫn lớn có thể không đáng giữ. Ngược lại, một mask rẻ hơn một chút có thể cho throughput tổng thể tốt hơn ngay cả khi acceptance thấp hơn. Đây là lý do đề tài tách representation score khỏi runtime score và kiểm chứng mối liên hệ giữa chúng.

8. ĐIỀU CHỈNH SPECULATIVE WINDOW K

8.1. Entropy

$$H(p) = -\sum_v p(v) \log p(v)$$

Entropy thấp thường tương ứng phân phối token tập trung hơn, còn entropy cao biểu thị nhiều lựa chọn cạnh tranh. Tuy nhiên, entropy không phải predictor hoàn hảo cho acceptance K bước tới. Do đó K sẽ không bị quyết định bởi một công thức cố định duy nhất; entropy chỉ là một feature của controller.

8.2. Acceptance-aware update

$$K_{t+1} = \text{clip}(K_t + \eta * \text{sign}(A_{\text{target}} - A_{\text{obs}}), K_{\text{min}}, K_{\text{max}})$$

Trong đó A_{obs} là acceptance quan sát trong các cycle gần nhất. Policy kết hợp entropy và acceptance history để tránh phản ứng quá mạnh với một token khó đơn lẻ.

8.3. Joint action

Thay vì quy tắc “entropy thấp $\rightarrow K=8$ ”, controller đánh giá các action candidate (S,K) bằng utility đo được. Ví dụ với mỗi action, hệ thống ước lượng số token kỳ vọng được commit, chi phí draft + verify và năng lượng dự kiến. Action có utility cao nhất dưới constraint được chọn.

9. HARDWARE- AND ENERGY-AWARE CONSTRAINED CONTROLLER (HECC)

9.1. State

Feature	Nguồn đo	Ý nghĩa
Entropy / top-k uncertainty	Target logits	Độ khó cục bộ của token
Acceptance history	Verifier	Độ phù hợp của draft gần đây
Context length	KV/cache manager	Ảnh hưởng attention latency và memory

Draft latency	CUDA events	Chi phí action
Verify latency	CUDA events	Chi phí validation
Peak VRAM	torch.cuda + NVML	Ràng buộc cứng
GPU utilization	NVML/CUPTI khi khả dụng	Mức bão hòa compute
Power / energy	NVML power sampling	Đánh giá J/token

9.2. Utility có ràng buộc

$$U = \lambda_1 * \text{expected_tokens_per_cycle} - \lambda_2 * \text{latency_cycle} - \lambda_3 * \text{energy_cycle} - \lambda_4 * \text{risk(VRAM)}$$

$$\begin{aligned} \text{risk(VRAM)} &= 0 && \text{if peakVRAM} \leq V_{\text{safe}} \\ &= \kappa * (\text{peakVRAM} - V_{\text{safe}})^2 && \text{otherwise} \end{aligned}$$

Có thể bổ sung penalty cho P95 TPOT hoặc jitter khi cần. Mục tiêu không phải “tối ưu mọi thứ cùng lúc” theo một scalar thần bí; controller cần hỗ trợ các mode rõ ràng như MAX-THROUGHPUT, MIN-ENERGY và SAFE-MEMORY.

9.3. Pseudocode

```

for each generation cycle:
    observe state s_t
    candidate_actions = {(S_i, K_j)}
    for a in candidate_actions:
        estimate utility U(s_t, a)
    reject a if estimated/observed peak VRAM > budget
    choose a* = argmax U(s_t, a)
    draft K tokens with S_i
    verify all candidates with full model
    commit accepted target KV
    update acceptance, latency, energy, VRAM statistics

```

10. THIẾT LẬP THỰC NGHIỆM

10.1. Hardware

Thành phần	Thiết lập
GPU	NVIDIA GeForce RTX 4050 Laptop GPU, 6GB GDDR6; 2,560 CUDA cores; 96-bit interface; nominal 6GB class [1]
CPU	Intel Core i7 hoặc AMD Ryzen 7 class; ghi rõ model thực tế trong report
RAM	Ghi lại dung lượng thực tế; không dùng như biến ẩn
OS	Ubuntu hoặc WSL2/Windows; cố định phiên bản cho reproduction
CUDA	CUDA 12.x, ghi rõ exact minor version
Framework	PyTorch 2.3+ / phiên bản chính xác tại thực nghiệm; Hugging Face Transformers
Optimization	torch.compile; FlashAttention/Triton chỉ bật khi benchmark có đối chứng
Telemetry	NVML/nvidia-smi; CUDA events; process RSS khi cần

10.2. Models

Qwen2.5-3B-Instruct có 36 transformer layers và hidden size 2048 theo model configuration; model card hiện hành mô tả context length lớn và generation limit 8K. [14] Llama-3.2-3B-Instruct có kiến trúc 3B-class với 28 transformer layers; cấu hình chính xác phải lấy trực tiếp từ config/checkpoint sử dụng để tránh ghi sai số liệu. [15]

10.3. Workloads

Nhóm	Benchmark	Mục đích
General text	CNN/DailyMail	Long-form summarization / common language
Code	HumanEval	Code generation, difficult local decisions
Math reasoning	GSM8K	Reasoning-heavy context
Dialogue	MT-Bench hoặc tập tương đương reproducible	Multi-turn generation
Synthetic long-context	Prompt với 2K/4K/8K/16K+ tùy khả năng VRAM	Kiểm tra context-dependent layer/attention cost

10.4. Experimental factors

- Precision: FP16/BF16 hoặc quantization configuration cố định và ghi rõ.
- Batch size: ưu tiên batch=1 để mô phỏng interactive edge inference; có thể thêm batch nhỏ làm robustness test.
- Context length: ít nhất 512, 2K, 4K và mức dài nhất vẫn chạy ổn định.
- Generation length: chuẩn hóa output length hoặc đo trên cùng prompt để tránh bias.
- Warm-up: tối thiểu 10 runs trước measurement; lặp nhiều seeds/runs cho confidence interval.

11. BASELINE VÀ ĐỐI CHIẾU

Baseline	Mục đích	Bắt buộc
Vanilla autoregressive	Mốc chuẩn không speculative	Có
Static layer skip	Tách hiệu quả layer skipping đơn giản khỏi controller	Có
Random layer skip	Đối chứng chất lượng selection	Có
CKA/static selected	Đánh giá giá trị của representation selection	Có
Fixed K self-spec	Đánh giá riêng adaptive K	Có
Adaptive K baseline	Đối chiếu entropy/acceptance policy	Có
CLaSp / KnapSpec / SpecBound / DEL khi khả thi	So sánh với hướng gần nhất; version và checkpoint phải cố định	Nên có
Standard speculative với draft model nhỏ	Đánh giá memory-speed trade-off	Nên có nếu vừa VRAM

Các baseline external phải dùng đúng code/checkpoint/hyperparameters công bố hoặc mô tả rõ khi không thể reproducing. Không được so sánh một phương pháp đã compile tối ưu với baseline chưa warm-up hoặc khác precision.

12. HỆ THỐNG CHỈ SỐ ĐÁNH GIÁ

$$\text{Speedup} = \text{Throughput_method} / \text{Throughput_vanilla}$$

$$\text{Acceptance} = \text{accepted_draft_tokens} / \text{total_draft_tokens}$$

$$\text{TPOT} = \text{total_generation_time} / \text{generated_tokens}$$

$$\text{Energy/token} = \int P(t)dt / \text{generated_tokens}$$

$$\text{Additional-VRAM} = \text{peakVRAM}(\text{method}) - \text{peakVRAM}(\text{vanilla baseline under matched setup})$$

TTFT phải được đo riêng với decode throughput. Peak VRAM phải dùng cùng định nghĩa giữa các baseline, ví dụ allocated, reserved và NVML process memory được báo cáo thành các cột riêng. Với RTX laptop, power draw là biến phụ thuộc TGP/thermal state; do đó báo cáo phải ghi exact GPU power setting nếu hệ thống cho phép.

13. ABLATION STUDY

Ablation	Câu hỏi
No CKA	Representation-aware selection có thực sự giúp không?
No hardware cost	Bỏ latency profile thì utility có suy giảm không?
Fixed K vs Adaptive K	Adaptive K có đóng góp độc lập không?
No power term	Energy-aware control có cải thiện J/token không?
No VRAM constraint	Peak VRAM có tăng khi tối ưu throughput thuần?
Open-loop vs closed-loop	Telemetry feedback có lợi hơn static/offline policy không?
Single objective vs Pareto	Một scalar utility có đủ hay Pareto frontier tốt hơn?
Short vs long context	Lợi ích có phụ thuộc context length không?

14. KẾT QUẢ KỲ VỌNG VÀ TIÊU CHÍ THÀNH CÔNG

Các số bên dưới là target nghiên cứu, không phải kết quả đã có. Chúng chỉ dùng để quyết định khi nào hypothesis được ủng hộ.

Chỉ số	Baseline expectation	Success criterion
Additional draft-model VRAM	>0 nếu dùng draft model độc lập	~0 extra model footprint
Greedy consistency	100% vanilla	100% ở cùng decoding condition
Distribution correctness	Cần exact sampler/verification	Không phát hiện lệch phân phối đáng kể ngoài sai số implementation
Speedup	1.0x	Có lợi ích có ý nghĩa thống kê ở ít nhất một workload
Acceptance	Phụ thuộc mask/K	Adaptive policy cải thiện hoặc giữ acceptance khi giảm compute
Peak VRAM	Phụ thuộc context	Không vượt hard budget trong safe mode
Energy/token	Phụ thuộc TGP	Pareto point tốt hơn baseline ở ít nhất một mode
Robustness	Policy tĩnh dễ suy giảm theo context	HECC không suy sụp khi context/load thay đổi

Không sử dụng ngưỡng “45-54 tok/s” hay “1.45-1.75x” như kết quả được khẳng định từ trước. Nếu thực nghiệm đạt thấp hơn, điều quan trọng là giải thích nguyên nhân bằng decomposition: draft cost, verify cost, acceptance, launch overhead, memory stalls và thermal behavior.

15. PHÂN TÍCH RỦI RO KỸ THUẬT

Rủi ro	Mức	Biện pháp
Draft mask có acceptance quá thấp	Cao	Khảo sát nhiều masks; giới hạn layer budget; dùng target hidden-state similarity và acceptance thật làm feedback
Python/framework overhead triệt tiêu speedup	Cao	CUDA events, torch.compile, CUDA graphs khi ổn định; profile kernel-level
KV cache bloat	Cao	Context sweep; allocator budget; có thể thêm KV quantization như nhánh phụ
Adaptive controller oscillation	Trung bình	Hysteresis, moving average, action cooldown, safety fallback
Power/thermal throttling	Trung bình	Warm-up dài, fixed power mode khi có thể, ghi TGP/temperature, lặp runs
External baseline khó reproducing	Trung bình	Chỉ claim comparison khi checkpoint/code hợp lệ; otherwise label as qualitative literature comparison
Novelty overlap với paper mới	Cao	Quét literature trước submission; ghi rõ candidate novelty, cập nhật related work sát ngày submit

16. KẾ HOẠCH TRIỂN KHAI 14 TUẦN

Tuần	Công việc	Deliverable
1-2	Baseline vanilla, measurement harness, CUDA events/NVML	Benchmark harness + baseline logs
3	Implement speculative verification/exactness tests	Unit tests + greedy/distribution checks
4-5	Implement static/random layer skip + layer profiling	Layer-cost/quality dataset
6	CKA/cosine/KL analysis	Candidate layer masks
7	Self-spec draft/verify stable on Qwen 3B	Working core pipeline
8	Fixed K and adaptive K baseline	K controller baseline
9	HECC v1: state/action/constraints	Runtime controller
10	Energy-aware + VRAM-safe modes	Pareto/constraint module
11	Long-context + thermal robustness	Stress-test report
12	Baselines CLaSp/KnapSpec/SpecBound/DEL where feasible	Comparative results
13	Ablation + statistical analysis	Tables/figures
14	Paper draft + reproducibility package + demo	Final report/paper repository

17. KIẾN TRÚC MÃ NGUỒN DỰ KIẾN

```

project/
  configs/
    qwen3b.yaml
    llama3_3b.yaml
  models/
    layer_skip_model.py
    cache_manager.py
    speculative/
      draft.py
      verify.py
    rejection_sampling.py
    controller/
      state.py
    action_space.py
    utility.py
    hecc.py
    pareto.py
    profiling/
      layer_cost.py
      cka.py
    latency_db.py
    telemetry/
      nvml.py
    cuda_events.py
    power.py
    benchmarks/
      generation.py
    human_eval.py
      gsm8k.py
      cnn_dm.py
      analysis/
        plots.py
        stats.py
      tests/
        test_exactness.py
        test_cache.py
        test_controller.py

```

Repository cần cố định commit hash, exact model revision, transformers/PyTorch/CUDA versions, launch command và seed. Mỗi experiment ghi cấu hình thành JSON/CSV để có thể tái lập và so sánh.

18. LỘ TRÌNH PHÁT TRIỂN THÀNH BÀI BÁO

18.1. Câu chuyện paper

Paper không nên kể theo thứ tự “chúng tôi viết một hệ thống rồi đo nó nhanh”. Câu chuyện nên bắt đầu từ constrained inference: trên consumer GPU 6GB, mục tiêu throughput đơn thuần có thể xung đột với memory headroom và energy. Self-speculative decoding loại bỏ draft model nhưng configuration vẫn phụ thuộc context và hardware. Đề tài đặt ra HECC để điều khiển configuration trong một vòng kín, rồi chứng minh trade-off bằng Pareto frontier.

18.2. Cấu trúc paper đề xuất

1. Introduction - problem, motivation, empirical observation on 6GB GPU.
2. Background - speculative decoding, self-speculative decoding and exactness.
3. Related Work - Draft & Verify, LayerSkip, CLaSp, DEL, SVIP, BanditSpec, KnapSpec, SpecBound, CAS-Spec, resource-aware serving.
4. Method - layer profiling, action space, HECC, constraints, Pareto modes.
5. Implementation - PyTorch/CUDA/NVML, cache handling, measurement methodology.
6. Experiments - speed, TPOT, VRAM, energy, long context, robustness.
7. Ablations - CKA, K, hardware cost, VRAM penalty, energy term, closed-loop.
8. Limitations - consumer GPU specificity, profile cost, external baseline reproducibility.
9. Conclusion - practical contribution and generalization hypothesis.

18.3. Tiêu chí để novelty đủ mạnh

- Có ít nhất một empirical finding không được các baseline hiện tại giải thích.
- HECC phải cải thiện một metric mục tiêu dưới cùng constraint, không chỉ thay đổi implementation.
- Ablation phải chứng minh từng thành phần đóng góp độc lập.
- Cần chỉ ra trường hợp policy thất bại và khi nào baseline tốt hơn; paper trung thực sẽ mạnh hơn.

19. VALIDATION CHO EXACTNESS VÀ ĐỘ ĐÚNG

19.1. Greedy

Với temperature=0/greedy, generate cùng prompt và cùng stopping condition bằng vanilla và speculative. So sánh toàn bộ sequence token-by-token. Nếu khác, lưu timestep đầu tiên bị divergence và dump p, q, chosen token, layer configuration để debug.

19.2. Sampling

Với sampling, kiểm tra cơ chế residual distribution. Có thể sử dụng statistical tests trên nhiều samples và các prompt cố định. Không diễn giải “output string giống nhau” như bằng chứng đầy đủ cho distribution preservation.

19.3. Cache invariants

- Target KV sau commit phải tương đương target vanilla tại các vị trí đã committed, trong tolerance phù hợp precision.
- Draft scratch state không được tồn tại như một canonical state sau cycle.
- Không có tensor dangling làm tăng peak reserved memory theo thời gian.

20. GIỚI HẠN VÀ PHẠM VI

Đề tài tập trung vào batch nhỏ/interactive inference trên GPU laptop 6GB. Kết quả có thể không đại diện cho A100/H100 hoặc server batching lớn. Một số kỹ thuật như CUDA Graphs, Triton và KV quantization có thể phụ thuộc phiên bản software/hardware. Vì vậy, claim chính nên là “hiệu quả trên resource-constrained consumer GPU” thay vì “tối ưu cho mọi GPU”.

Đề tài cũng không nhằm huấn luyện lại LLM. Nếu cần calibration data, đây chỉ là profiling/analysis, không phải fine-tuning target model. Nếu về sau cần một training-based variant, nó phải được tách thành extension để không làm thay đổi core question của bản paper chính.

21. SẢN PHẨM ĐẦU RA

Đầu ra	Mô tả
Core implementation	Self-speculative draft/verify + exact verification
Controller	HECC constrained runtime controller + Pareto modes
Benchmark harness	Reproducible speed/VRAM/energy/quality measurement
Analysis	Layer similarity, latency model, acceptance, Pareto frontier
Demo	Local web UI/CLI chạy trên laptop 6GB
Thesis	Báo cáo đầy đủ về methodology, results, limitations
Paper	Conference/journal-style manuscript; novelty claim chỉ sau khi complete literature search + experiments
Artifact	Config, seed, logs, scripts, environment file, exact checkpoint revision

22. TÀI LIỆU THAM KHẢO CHÍNH

- [1] NVIDIA. GeForce RTX 40 Series Gaming Laptops / Compare GeForce RTX Laptops. Thông số RTX 4050 Laptop GPU: 6GB GDDR6, 2560 CUDA cores, 96-bit interface, configurable laptop power. <https://www.nvidia.com/en-us/geforce/laptops/40-series/>
- [2] Leviathan, Y., Kalman, M., & Matias, Y. (2023). Fast Inference from Transformers via Speculative Decoding. ICML 2023, PMLR 202:19274-19286. <https://proceedings.mlr.press/v202/leviathan23a.html>
- [3] Zhang, J., Wang, J., Li, H., Shou, L., Chen, K., Chen, G., & Mehrotra, S. (2024). Draft & Verify: Lossless Large Language Model Acceleration via Self-Speculative Decoding. ACL 2024. <https://aclanthology.org/2024.acl-long.607/>
- [4] Elhoushi, M. et al. (2024). LayerSkip: Enabling Early Exit Inference and Self-Speculative Decoding. ACL 2024. <https://aclanthology.org/2024.acl-long.681/>
- [5] Chen, L. et al. (2025). CLaSp: In-Context Layer Skip for Self-Speculative Decoding. ACL 2025. <https://aclanthology.org/2025.acl-long.1525/>
- [6] Entezari Zarch, H., Gao, L., Jiang, C., & Annavaram, M. (2025). DEL: Context-Aware Dynamic Exit Layer for Efficient Self-Speculative Decoding. arXiv:2504.05598. <https://arxiv.org/abs/2504.05598>
- [7] Zhang, Z. et al. (2025). Draft Model Knows When to Stop: Self-Verification Length Policy for Long-Context Speculative Decoding. EMNLP 2025. <https://aclanthology.org/2025.emnlp-main.844/>

- [8] Hou, Y. et al. (2025). BanditSpec: Adaptive Speculative Decoding via Bandit Algorithms. ICML 2025, PMLR 267. <https://proceedings.mlr.press/v267/hou25h.html>
- [9] Liu, J., Park, B., & Shen, X. (2025). A Drop-In Solution for On-the-Fly Adaptation of Speculative Decoding in Large Language Models. ACL 2025. <https://aclanthology.org/2025.acl-long.482/>
- [10] Wen, Z., & Feng, Y. (2026). SpecBound: Adaptive Bounded Self-Speculation with Layer-wise Confidence Calibration. Findings of ACL 2026. <https://aclanthology.org/2026.findings-acl.802/>
- [11] Cha, S., Kim, G., Han, D., Yang, T., & Han, I. (2026). KnapSpec: Self-Speculative Decoding via Adaptive Layer Selection as a Knapsack Problem. arXiv:2602.20217. <https://arxiv.org/abs/2602.20217>
- [12] Ning, Z. et al. (2025). CAS-Spec: Cascade Adaptive Self-Speculative Decoding for On-the-Fly Lossless Inference Acceleration of LLMs. NeurIPS 2025. https://papers.neurips.cc/paper_files/paper/2025/hash/5e27f5d8ecd30ac6235892ef97676e78-Abstract-Conference.html
- [13] Nightjar: Dynamic adaptive speculative decoding for large language models serving. Journal of Systems Architecture, Vol. 178, September 2026, 103889. <https://doi.org/10.1016/j.sysarc.2026.103889>
- [14] Qwen Team. Qwen2.5-3B-Instruct model card. Hugging Face. <https://huggingface.co/Qwen/Qwen2.5-3B-Instruct>
- [15] Meta AI. Llama 3.2 model documentation/model card. Exact checkpoint configuration should be frozen in the experiment manifest.
- [16] Cai, T. et al. (2024). Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads. ICML 2024, PMLR 235. <https://proceedings.mlr.press/v235/cai24b.html>
- [17] Li, Y., Wei, F., Zhang, C., & Zhang, H. (2024). EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty. ICML 2024, PMLR 235. <https://proceedings.mlr.press/v235/li24bt.html>

23. KẾT LUẬN ĐỀ CƯƠNG

Đề tài được định hướng thành một nghiên cứu về hệ thống suy luận LLM có ràng buộc tài nguyên, thay vì chỉ là một implementation của speculative decoding. Phần cốt lõi là self-speculative decoding training-free trên GPU 6GB; phần nghiên cứu là tìm hiểu mối quan hệ giữa representation similarity, acceptance, context-dependent latency, peak VRAM và energy/token; phần ứng viên novelty là HECC - một controller vòng kín tối ưu cấu hình (layer budget, K) dưới constraints thực đo của thiết bị.

Để có khả năng phát triển thành paper, điều kiện tiên quyết là không định trước kết quả và không claim novelty dựa trên thuật ngữ. Paper chỉ nên chốt contribution sau khi thực nghiệm chứng minh một trade-off mới hoặc một finding mới so với các công trình 2024-2026. Nếu HECC chỉ tái tạo kết quả của KnapSpec/DEL/SpecBound dưới một tên mới, hướng nghiên cứu phải tiếp tục điều chỉnh. Ngược lại, nếu constrained Pareto control cho thấy lợi ích rõ ràng trên consumer 6GB GPU và có ablation đầy đủ, đây có thể là một câu chuyện paper hợp lý trong Efficient Inference / Systems for AI.

Trong trao đổi với giáo sư, đề tài nên được trình bày thành ba lớp: nền tảng literature đã biết; hypothesis/candidate contribution cần kiểm chứng; và implementation/benchmark. Target speedup không được trình bày như kết quả thực nghiệm, còn novelty chỉ được chốt sau khi hoàn thành literature review cập nhật và ablation study.